



Time Series Anomaly Detection Methods: ARIMA, FFT, GARCH, Wavelets

Roadmap

A structured progression of methods in the `harbinger` package (R) — from the simplest baseline to deep architectures and multivariate analysis.

01

Fundamentals

Histogram baseline

03

Supervised

Regression and ML classification

05

Ensembles & Deep

Autoencoders, VAE, AAE

02

Residual Models

ARIMA, FFT, EMD, Wavelet...

04

Clustering

k-means, DTW

06

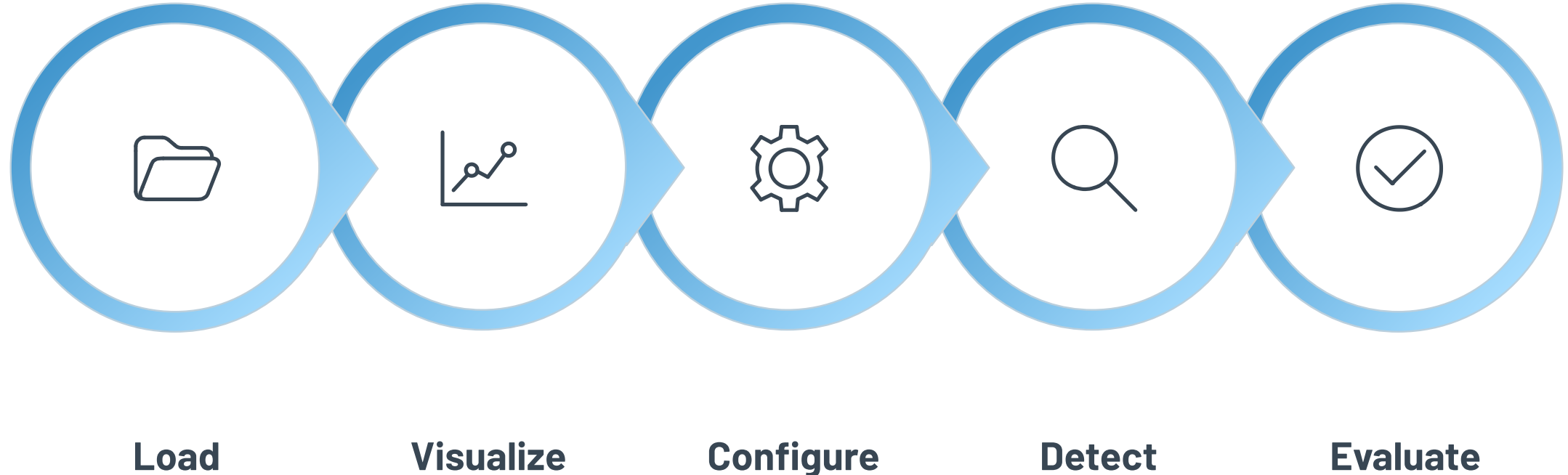
Multivariate

PCA for multiple dimensions



Run the full examples and explore the source code at: <https://github.com/cefet-rj-dal/harbinger/tree/master/examples>

Standard Harbinger Workflow



Every detector in Harbinger follows this 5-step workflow. The consistency of the interface makes it easy to compare methods from completely different families with minimal code changes — only the model constructor needs to change.

Histogram – Empirical Distribution Baseline

How it works

- Builds a frequency histogram from the observed values of the series
- Bins with very low density (below a configurable threshold) are marked as anomalous regions
- Any observation falling in a low-density bin or outside the observed range is flagged as an anomaly
- No model is fitted — the method is purely data-driven

When to use

- As the very first baseline before any model-based approach
- When no assumptions about the temporal structure of the series can be made
- Fast, parameter-light, and fully interpretable

Key parameters

- `density_threshold` (default 0.05): minimum bin density below which a region is considered anomalous

Reference

No specific academic citation — this is a classical non-parametric baseline. See harbinger documentation: <https://cran.r-project.org/web/packages/harbinger>

📄 Anomaly score: bin density of the observed value. Lower density → higher anomaly score.

01 · hanr_histogram() – Histogram Baseline

```
# Instalar (se necessário)
# install.packages("harbinger")

library(daltoolbox)
library(harbinger)

# Carregar datasets de exemplo
data(examples_anomalies)

# Selecionar série simples
dataset <- examples_anomalies$simple
head(dataset)

## serie event
## 1 1.0000000 FALSE
## 2 0.9689124 FALSE

# Visualizar série bruta
har_plot(harbinger(),
  dataset$serie,
  event = dataset$event)

# Configurar detector
model <- hanr_histogram(
  density_threshold = 0.05)
```

```
# Fit (no-op, mas mantém workflow)
model <- fit(model, dataset$serie)

# Detectar
detection <- detect(model,
  dataset$serie)

# Ver anomalias
print(detection |>
  dplyr::filter(event == TRUE))
## idx event type
## 1 1 TRUE anomaly
## 2 50 TRUE anomaly

# Avaliar
evaluation <- evaluate(model,
  detection$event, dataset$event)
print(evaluation$confMatrix)
## detection TRUE FALSE
## TRUE 1 1
## FALSE 0 99

# Plotar resultado final
har_plot(model, dataset$serie,
  detection, dataset$event)
```

Step-by-step

- Load `daltoolbox` and `harbinger`
- Select `examples_anomalies$simple`
- Visualize the raw series before fitting any model
- Configure with `density_threshold = 0.05`
- `fit()` is a no-op here, but preserves the standard workflow pattern
- `detect()` returns a dataframe with an `event` column
- Evaluate with `evaluate()` and inspect the `confMatrix`
- Plot detections overlaid on the original series

📌 **Key concept:** `hanr_histogram()` builds a histogram of the observed values and flags as anomalies any points that fall in low-density bins or outside the observed range. It serves as a useful baseline since it focuses solely on the empirical distribution.

ARIMA – Autoregressive Integrated Moving Average

How it works

- Models the series as a linear combination of its own past values (AR), past errors (MA), and differencing for stationarity (I)
- `fit()` automatically selects the order (p,d,q) and estimates parameters
- `detect()` computes standardized residuals: $(\text{observed} - \text{predicted}) / \sigma$
- Residuals exceeding a threshold (via `harutils()`) are flagged as anomalies

When to use

- Series with linear autocorrelation structure
- Stationary or easily differenced series
- When a well-established statistical model is preferred

Key parameters

- No required parameters — automatic order selection via `auto.arima()`
- Threshold controlled internally by `harutils()`

Reference

Box, G.E.P. & Jenkins, G.M. (1976). Time Series Analysis: Forecasting and Control. Holden-Day.

📄 Anomaly score: $|\text{standardized residual}|$.
Large deviations from the ARIMA forecast signal anomalies.

02 · hanr_arima() – ARIMA Residuals

```
library(daltoolbox)
library(harbinger)

data(examples_anomalies)
dataset <- examples_anomalies$simple

# Visualizar série bruta
har_plot(harbinger(), dataset$serie)

# Configurar detector ARIMA
model <- hanr_arima()

# Ajustar (estima ordem ARIMA
# e armazena parâmetros)
model <- fit(model, dataset$serie)

# Detectar (calcula resíduos
# padronizados e flags)
detection <- detect(model,
  dataset$serie)
```

```
print(detection |>
  dplyr::filter(event == TRUE))
## idx event type
## 1 50 TRUE anomaly

# Avaliar
evaluation <- evaluate(model,
  detection$event, dataset$event)
print(evaluation$confMatrix)
## detection TRUE FALSE
##   TRUE  1   0
##  FALSE  0 100

# Plotar detecções vs ground truth
har_plot(model, dataset$serie,
  detection, dataset$event)

# Plotar magnitude dos resíduos
# e limiares de decisão
har_plot(model,
  attr(detection, "res"),
  detection, dataset$event,
  yline = attr(detection,
    "threshold"))
```

Step by step

- Same workflow: load data, visualize, configure, fit, detect
- `hanr_arima()` with no arguments uses automatic order selection
- `fit()` estimates ARIMA(p,d,q) and stores the parameters
- `detect()` computes standardized residuals and applies a threshold via `harutils()`
- Perfect result on this dataset: 1 TP, 0 FP
- The second `har_plot()` displays residual magnitudes with the decision threshold line

📌 **Key concept:** Fits an ARIMA(p,d,q) model to the series and uses large standardized residuals as evidence of an anomaly. Thresholds are defined via `harutils()`.

Residual Methods Family – Overview



ARIMA

Fits a statistical model and uses standardized residuals for anomaly detection.



FBIAD

Compares forward and backward inertia in sliding windows to identify deviations.



EMD/REMD

Decomposes the series into empirical modes and analyzes components for anomalies.



FFT + Changepoint

Filters the power spectrum and detects change points to identify anomalies.



Key concept: All methods share the same workflow: `fit()` → `detect()` → `evaluate()`. The difference lies in how each one computes the anomaly score.

FBIAD – Forward-Backward Inertia Anomaly Detection

How it works

- For each point, computes the mean of a forward window (future values) and a backward window (past values)
- The inertia score measures how much the point deviates from both local trends simultaneously
- A point is flagged only if it breaks both the forward and backward inertia — reducing false positives
- Scores are aggregated and thresholded via `harutils()`

When to use

- Series with smooth local trends where abrupt point anomalies stand out
- When you want a method that is robust to one-sided trend changes
- Good complement to ARIMA in an ensemble

Key parameters

- No required parameters — window size is determined automatically
- Threshold controlled by `harutils()`

Reference

Lima, T. et al. (2022). FBIAD: Forward-backward inertia anomaly detection. IJCNN 2022.

- ❏ Anomaly score: combined deviation from forward and backward local means. Both must be exceeded for a flag.

03 · hanr_fbiad() – Forward-Backward Inertia Anomaly Detection

```
library(daltoolbox)
library(harbinger)

data(examples_anomalies)
dataset <- examples_anomalies$simple

har_plot(harbinger(), dataset$serie)

# Configurar FBIAD
model <- hanr_fbiad()
model <- fit(model, dataset$serie)

detection <- detect(model,
                    dataset$serie)

print(detection |>
      dplyr::filter(event == TRUE))
## idx event type
## 1 50 TRUE anomaly
```

```
evaluation <- evaluate(model,
  detection$event, dataset$event)
print(evaluation$confMatrix)
## detection TRUE FALSE
##   TRUE  1   0
##   FALSE 0 100

har_plot(model, dataset$serie,
  detection, dataset$event)

har_plot(model,
  attr(detection, "res"),
  detection, dataset$event,
  yline = attr(detection,
    "threshold"))
```

Step-by-step

- FBIAD = Forward and Backward Inertial Anomaly Detector
- Compares each point against the means of forward and backward sliding windows
- Flags observations that break both temporal trends simultaneously
- Scores are summarized and thresholded via `harutils()`
- Result: 1 TP, 0 FP on this dataset

📌 **Key concept:** FBIAD compares each point against its forward and backward inertia, flagging observations that simultaneously break both temporal trends. Ref: Lima et al., IJCNN 2022.

EMD – Empirical Mode Decomposition

How it works

- Decomposes the series into a set of Intrinsic Mode Functions (IMFs) using the sifting algorithm — no prior basis functions required
- harbinger uses CEEMD (Complementary Ensemble EMD) via the hht package for improved noise robustness
- High-frequency IMFs capture abrupt local deviations; anomalies appear as elevated energy in these components
- The aggregated high-frequency signal is thresholded via harutils()

When to use

- Non-stationary and non-linear series where ARIMA assumptions do not hold
- When the series has multiple oscillatory modes at different scales
- As an alternative to FFT when the frequency content varies over time

Key parameters

- No required parameters — CEEMD decomposition is automatic
- Threshold controlled by harutils()

Reference

Huang, N.E. et al. (1998). The empirical mode decomposition and the Hilbert spectrum for nonlinear and non-stationary time series analysis. Proc. R. Soc. London A, 454, 903–995.

❏ Anomaly score: energy of high-frequency IMF components. Abrupt deviations concentrate energy in the finest modes.

04 · hanr_emd() – Empirical Mode Decomposition

```
library(daltoolbox)
library(harbinger)

data(examples_anomalies)
dataset <- examples_anomalies$simple

har_plot(harbinger(), dataset$serie)

# Configurar detector EMD (CEEMD)
model <- hanr_emd()
model <- fit(model, dataset$serie)

detection <- detect(model,
                    dataset$serie)

print(detection |>
      dplyr::filter(event == TRUE))
## idx event type
## 1 50 TRUE anomaly
```

```
evaluation <- evaluate(model,
                      detection$event, dataset$event)
print(evaluation$confMatrix)
## detection TRUE FALSE
##   TRUE  1   0
##   FALSE 0 100

har_plot(model, dataset$serie,
          detection, dataset$event)

har_plot(model,
          attr(detection, "res"),
          detection, dataset$event,
          yline = attr(detection,
                      "threshold"))
```

Step-by-step

- Uses CEEMD (Complementary Ensemble EMD) via `hht::CEEMD`
- Decomposes the series into IMFs (Intrinsic Mode Functions)
- Isolates high-frequency components that capture abrupt deviations
- Anomalies are flagged where these components exhibit large magnitude
- Thresholds are applied via `harutils()`

❏ **Key concept:** EMD decomposes the series into IMFs and isolates high-frequency components. Abrupt deviations appear as elevated energy in these components. Ref: Huang et al., 1998.

FFT – Spectral Filtering & Changepoint Variants

How it works

- Applies the Fast Fourier Transform to decompose the series into frequency components
- A high-pass filter removes low-frequency (normal) components; the residual captures abrupt deviations
- The frequency cutoff can be selected by different strategies — giving rise to four variants:
 - `hanr_fft()`: fixed cutoff based on power spectrum
 - `hanr_fft_amoc()`: AMOC (At Most One Change) selects a single changepoint in the spectrum
 - `hanr_fft_binseg()`: Binary Segmentation recursively finds the optimal cutoff
 - `hanr_fft_amoc_cusum()` / `hanr_fft_binseg_cusum()`: CUSUM accumulation applied before changepoint selection
 - `hanr_fft_sma()`: dominant frequency drives an adaptive moving average; residual = original – smoothed

When to use

- Series with periodic or quasi-periodic structure
- When the anomaly manifests as a high-frequency spike against a smooth background

Key parameters

- No required parameters for any variant
- Cutoff strategy differs per variant (see above)
- Threshold via `harutils()`

Reference

Cooley, J.W. & Tukey, J.W. (1965). An algorithm for the machine calculation of complex Fourier series. Mathematics of Computation, 19(90), 297–301.

📄 Anomaly score: magnitude of the high-frequency residual after spectral filtering. All variants share the same `fit()` → `detect()` interface.

05 · `hanr_fft()` – Spectral Filtering via FFT

```
library(daltoolbox)
library(harbinger)

data(examples_anomalies)
dataset <- examples_anomalies$simple

har_plot(harbinger(), dataset$serie)

# Configurar detector FFT
model <- hanr_fft()

# Fit: sem treinamento, rastreia
# parâmetros espectrais
model <- fit(model, dataset$serie)

# Detectar: calcula resíduos
# de alta frequência e flags
detection <- detect(model,
  dataset$serie)

print(detection |>
  dplyr::filter(event == TRUE))
## idx event type
## 1 50 TRUE anomaly
```

```
evaluation <- evaluate(model,
  detection$event, dataset$event)
print(evaluation$confMatrix)
## detection TRUE FALSE
##   TRUE  1   0
##   FALSE 0 100

har_plot(model, dataset$serie,
  detection, dataset$event)

har_plot(model,
  attr(detection, "res"),
  detection, dataset$event,
  yline = attr(detection,
    "threshold"))
```

Step-by-step

- Applies a high-pass filter via the Discrete Fourier Transform
- Computes the spectrum using FFT and selects the cutoff based on the power spectrum
- Inverse FFT reconstructs the high-frequency signal component
- Residual magnitudes are summarized and thresholded via `harutils()`
- Result: 1 TP, 0 FP

📌 **Key concept:** FFT applies a high-pass filter in the frequency domain. Low-frequency components (normal behavior) are removed; what remains captures abrupt deviations — potential anomalies.

06 · hanr_fft_amoc() – FFT with AMOC Selection

```
library(daltoolbox)
library(harbinger)

data(examples_anomalies)
dataset <- examples_anomalies$simple

har_plot(harbinger(), dataset$serie)

# FFT + AMOC
model <- hanr_fft_amoc()
model <- fit(model, dataset$serie)

detection <- detect(model,
                    dataset$serie)
# Resultado: idx=50, 1 TP, 0 FP

print(detection |>
      dplyr::filter(event == TRUE))
## idx event type
## 1 50 TRUE anomaly
```

```
evaluation <- evaluate(model,
                      detection$event, dataset$event)
print(evaluation$confMatrix)
## detection TRUE FALSE
##   TRUE  1   0
##   FALSE 0 100

har_plot(model, dataset$serie,
          detection, dataset$event)
har_plot(model,
          attr(detection, "res"),
          detection, dataset$event,
          yline = attr(detection,
                      "threshold"))
```

Step-by-step – FFT + AMOC

- AMOC (At Most One Change) automatically selects the cutoff point in the power spectrum
- Frequencies below the AMOC changepoint are removed
- Anomalies are flagged based on the resulting residual magnitudes
- Perfect result: 1 TP, 0 FP

 **Key concept:** AMOC detects a single change point in the power spectrum and uses it as the frequency cutoff, filtering out low-frequency components before computing residuals.

07 · hanr_fft_amoc_cusum() – FFT + CUSUM + AMOC

```
library(daltoolbox)
library(harbinger)

data(examples_anomalies)
dataset <- examples_anomalies$simple

# FFT + AMOC + CUSUM
model <- hanr_fft_amoc_cusum()
model <- fit(model, dataset$serie)

detection <- detect(model,
                    dataset$serie)
```

```
print(detection |>
      dplyr::filter(event == TRUE))
## idx event type
## 1 50 TRUE anomaly

evaluation <- evaluate(model,
                        detection$event, dataset$event)
print(evaluation$confMatrix)
## detection TRUE FALSE
##   TRUE  1   0
##  FALSE  0 100

har_plot(model, dataset$serie,
          detection, dataset$event)
har_plot(model,
          attr(detection, "res"),
          detection, dataset$event,
          yline = attr(detection,
                       "threshold"))
```

Step-by-step – FFT + AMOC + CUSUM

- CUSUM transforms the power spectrum to emphasize gradual changes
- AMOC selects a single changepoint as the frequency cutoff
- Pipeline: FFT → CUSUM on spectrum → AMOC → inverse FFT → threshold
- Perfect result: 1 TP, 0 FP

📌 **Key concept:** CUSUM accumulates deviations in the spectrum before AMOC, amplifying gradual transitions. This combination is more robust for series with smooth regime changes.

08 · hanr_fft_binseg() – FFT with Binary Segmentation

```
library(daltoolbox)
library(harbinger)

data(examples_anomalies)
dataset <- examples_anomalies$simple

har_plot(harbinger(), dataset$serie)

# FFT + BinSeg
model <- hanr_fft_binseg()
model <- fit(model, dataset$serie)

detection <- detect(model,
                    dataset$serie)
# Resultado: idx=50, 1 TP, 0 FP
```

```
print(detection |>
      dplyr::filter(event == TRUE))
## idx event type
## 1 50 TRUE anomaly

evaluation <- evaluate(model,
                        detection$event, dataset$event)
print(evaluation$confMatrix)
## detection TRUE FALSE
##   TRUE  1   0
##  FALSE  0 100

har_plot(model, dataset$serie,
          detection, dataset$event)
har_plot(model,
          attr(detection, "res"),
          detection, dataset$event,
          yline = attr(detection,
                       "threshold"))
```

Step-by-step – FFT + BinSeg

- Binary Segmentation (BinSeg) selects the cutoff point in the power spectrum
- Frequencies below the detected changepoint are suppressed
- Anomalies are flagged based on residual magnitude via `harutils()`
- Perfect result: 1 TP, 0 FP

📌 **Key concept:** BinSeg recursively splits the spectrum to find the optimal frequency cutoff point. It serves as an alternative to AMOC when multiple potential changepoints are present.

09 · hanr_fft_binseg_cusum() – FFT + CUSUM + BinSeg

```
library(daltoolbox)
library(harbinger)

data(examples_anomalies)
dataset <- examples_anomalies$simple

# FFT + CUSUM + BinSeg
model <- hanr_fft_binseg_cusum()
model <- fit(model, dataset$serie)

detection <- detect(model,
                    dataset$serie)
```

```
print(detection |>
      dplyr::filter(event == TRUE))
## idx event type
## 1 50 TRUE anomaly

evaluation <- evaluate(model,
                        detection$event, dataset$event)
print(evaluation$confMatrix)
## detection TRUE FALSE
##   TRUE  1   0
##   FALSE 0 100

har_plot(model, dataset$serie,
          detection, dataset$event)
har_plot(model,
          attr(detection, "res"),
          detection, dataset$event,
          yline = attr(detection,
                       "threshold"))
```

Step-by-step – FFT + CUSUM + BinSeg

- CUSUM is applied to the power spectrum before BinSeg
- Low-frequency energy below the detected cutoff is removed
- Pipeline: FFT → CUSUM → BinSeg → Inverse FFT → threshold
- Perfect result: 1 TP, 0 FP

 **Key concept:** Combines CUSUM accumulation with binary segmentation. Ideal for series where spectral change is gradual and there are multiple changepoint candidates.

10 · hanr_fft_sma() – FFT + Adaptive Moving Average

```
library(daltoolbox)
library(harbinger)

data(examples_anomalies)
dataset <- examples_anomalies$simple

# Configurar FFT + SMA
model <- hanr_fft_sma()
model <- fit(model, dataset$serie)

# Detectar (aviso esperado sobre
# comprimento do objeto)
detection <- detect(model,
                    dataset$serie)
## Warning: longer object length is
## not a multiple of shorter object

print(detection |>
      dplyr::filter(event == TRUE))
## idx event type
## 1 50 TRUE anomaly
```

```
evaluation <- evaluate(model,
  detection$event, dataset$event)
print(evaluation$confMatrix)
## detection TRUE FALSE
##   TRUE  1   0
##   FALSE 0 100

har_plot(model, dataset$serie,
  detection, dataset$event)

har_plot(model,
  attr(detection, "res"),
  detection, dataset$event,
  yline = attr(detection,
    "threshold"))
```

Step-by-step – FFT + Adaptive SMA

- Estimates the dominant frequency from the FFT spectrum
- Uses that frequency to define the smoothing window for the moving average (SMA)
- Computes residuals: original series minus the smoothed series
- Large residuals are flagged as anomalies via harutils()
- The object length warning is expected and does not affect the result

📌 **Key concept:** FFT + adaptive SMA: the dominant spectral frequency automatically determines the smoothing window size. Residuals (original – smoothed) capture abrupt deviations from the expected pattern.

GARCH – Generalized Autoregressive Conditional Heteroscedasticity

How it works

- Models the conditional variance of the series as a function of past squared residuals (ARCH terms) and past variances (GARCH terms)
- `harbinger` uses `sGARCH(1,1)` with an ARMA mean equation by default
- `fit()` estimates both the mean and variance equations simultaneously
- `detect()` computes standardized residuals: $(\text{observed} - \text{mean}) / \text{conditional_std}$
- Points with large standardized residuals — exceeding what the local volatility predicts — are flagged

When to use

- Series with time-varying volatility (volatility clustering)
- Financial or sensor data where variance changes over time
- When anomalies manifest as spikes relative to the local noise level, not the global mean

Key parameters

- No required parameters — `sGARCH(1,1)` with ARMA mean is the default
- Threshold via `harutils()`

References

Engle, R.F. (1982). Autoregressive conditional heteroscedasticity with estimates of the variance of United Kingdom inflation. *Econometrica*, 50(4), 987–1007.

Bollerslev, T. (1986). Generalized autoregressive conditional heteroscedasticity. *Journal of Econometrics*, 31(3), 307–327.

 **Anomaly score:** $|\text{standardized residual}|$ relative to the conditional variance. Captures spikes that are large given the local volatility.

11 · hanr_garch() – Conditional Heteroscedasticity

```
library(daltoolbox)
library(harbinger)

data(examples_anomalies)
dataset <- examples_anomalies$simple

# Configurar detector GARCH
# Configuração típica: sGARCH(1,1)
# com média ARMA
model <- hanr_garch()
model <- fit(model, dataset$serie)

detection <- detect(model,
                    dataset$serie)

print(detection |>
      dplyr::filter(event == TRUE))
## idx event type
## 1 50 TRUE anomaly
```

```
evaluation <- evaluate(model,
  detection$event, dataset$event)
print(evaluation$confMatrix)
## detection TRUE FALSE
##   TRUE  1   0
##   FALSE 0 100

har_plot(model, dataset$serie,
  detection, dataset$event)

har_plot(model,
  attr(detection, "res"),
  detection, dataset$event,
  yline = attr(detection,
    "threshold"))
```

Step by step

- Fits a GARCH model to capture conditional heteroscedasticity
- Default configuration: sGARCH(1,1) with an ARMA mean equation
- Observations with large standardized residuals are flagged as anomalies
- Residuals are summarized and thresholded via `harutils()`
- Best suited for series with time-varying volatility

📌 **Key concept:** GARCH models the conditional variance of the series. Anomalies are points with large standardized residuals — deviations that exceed what the local volatility would predict. Refs: Engle 1982, Bollerslev 1986.

REMD – Robust EMD with ARIMA

How it works

- Combines EMD decomposition with ARIMA residual modeling in a two-stage pipeline
- Stage 1: CEEMD decomposes the series into IMFs, capturing non-stationary oscillatory structure
- Stage 2: ARIMA models the dynamics of the residual component left after EMD
- Anomalies are flagged where the combined residual (EMD + ARIMA) exceeds the threshold
- The combination makes the detector more robust than either method alone

When to use

- Non-stationary series with both oscillatory modes and autocorrelated residuals
- When `hanr_emd()` alone produces too many false positives
- When `hanr_arima()` alone misses anomalies masked by non-linear oscillations

Key parameters

- No required parameters — both stages are automatic
- Threshold via `harutils()`
- Note: may produce 1 FP on simple datasets due to higher sensitivity

Reference

Souza, V. et al. (2024). REMD: Robust time series event detection combining EMD and ARIMA residuals. IJCNN 2024.

📄 **Anomaly score:** residual after both EMD decomposition and ARIMA modeling. Higher sensitivity than either method alone — threshold tuning may be needed.

12 · hanr_remd() – Robust EMD with ARIMA

```
library(daltoolbox)
library(harbinger)

data(examples_anomalies)
dataset <- examples_anomalies$simple

har_plot(harbinger(), dataset$serie)

# Configurar detector REMD
model <- hanr_remd()
model <- fit(model, dataset$serie)

detection <- detect(model,
                    dataset$serie)

print(detection |>
      dplyr::filter(event == TRUE))
## idx event type
## 1 50 TRUE anomaly
## 2 54 TRUE anomaly
```

```
evaluation <- evaluate(model,
                      detection$event, dataset$event)
print(evaluation$confMatrix)
## detection TRUE FALSE
##   TRUE  1   1
##   FALSE 0  99

har_plot(model, dataset$serie,
          detection, dataset$event)

har_plot(model,
          attr(detection, "res"),
          detection, dataset$event,
          yline = attr(detection,
                      "threshold"))
```

Step by step

- REMD combines EMD with ARIMA modeling
- IMFs capture the non-stationary structure of the series
- ARIMA models the residual dynamics after decomposition
- Large residual magnitudes flag anomalies via `harutils()`
- Detected 2 events (idx 50 and 54): 1 TP, 1 FP on this dataset

📌 **Key concept:** REMD combines EMD (captures non-stationary structure via IMFs) with ARIMA (models residual dynamics). The combination is more robust than either method in isolation. Ref: Souza et al., IJCNN 2024.

RTAD – Residual Time-series Anomaly Detector

How it works

- Adapts to local dynamics using components derived from EMD decomposition
- Estimates robust dispersion (e.g., MAD — Median Absolute Deviation) within sliding windows
- Each point is scored relative to the local expected behavior, not a global model
- Points that deviate significantly from the local adaptive model are flagged
- Thresholds are computed via harutils()

When to use

- Series with non-stationary local behavior (mean or variance shifts over time)
- When global thresholds (as in ARIMA or FFT) produce too many false positives
- As a more flexible alternative to FBIAD for series with irregular local structure

Key parameters

- No required parameters — window size and dispersion estimation are automatic
- Threshold via harutils()

Reference

Based on robust statistics principles. See harbinger documentation and: Rousseeuw, P.J. & Croux, C. (1993). Alternatives to the median absolute deviation. Journal of the American Statistical Association, 88(424), 1273–1283.



Anomaly score: deviation from the local adaptive model, normalized by robust dispersion. More flexible than fixed global thresholds.

13 · hanr_rtad() – Adaptive Window-Based Detector

```
library(daltoolbox)
library(harbinger)

data(examples_anomalies)
dataset <- examples_anomalies$simple

har_plot(harbinger(), dataset$serie)

# Configurar RTAD
model <- hanr_rtad()
model <- fit(model, dataset$serie)

detection <- detect(model,
                    dataset$serie)

print(detection |>
      dplyr::filter(event == TRUE))
## idx event type
## 1 50 TRUE anomaly
```

```
evaluation <- evaluate(model,
  detection$event, dataset$event)
print(evaluation$confMatrix)
## detection TRUE FALSE
##   TRUE  1   0
##   FALSE 0 100

har_plot(model, dataset$serie,
  detection, dataset$event)

har_plot(model,
  attr(detection, "res"),
  detection, dataset$event,
  yline = attr(detection,
    "threshold"))
```

Step-by-step

- RTAD adapts to local dynamics using components derived from EMD
- Uses robust dispersion estimation within sliding windows
- Points with large deviations relative to the modeled behavior are flagged
- Adaptive thresholds computed via `harutils()`
- Perfect result on this dataset: 1 TP, 0 FP

📌 **Key concept:** RTAD (Residual Time-series Anomaly Detector) adapts to local dynamics using EMD components and robust dispersion within sliding windows — making it more flexible than fixed global thresholds.

Wavelet – Multiresolution Signal Analysis

How it works

- Decomposes the series into multiple resolution levels using the Maximal Overlap Discrete Wavelet Transform (MODWT)
- Each level captures oscillations at a different time scale — from fine (high-frequency) to coarse (low-frequency)
- Anomalies appear as elevated localized energy in the high-frequency detail coefficients
- Detail coefficients are aggregated into a magnitude signal and thresholded via `harutils()`
- Unlike FFT, wavelets preserve temporal localization — they reveal when and where in the series the anomaly occurs

When to use

- Series with transient or localized anomalies (spikes, bursts)
- When both frequency content and temporal location of the anomaly matter
- Non-stationary series where FFT's global frequency assumption is too rigid

Key parameters

- No required parameters — MODWT decomposition is automatic
- Threshold via `harutils()`

Reference

Mallat, S.G. (1989). A theory for multiresolution signal decomposition: the wavelet representation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 11(7), 674–693.

📌 **Anomaly score:** localized energy in high-frequency wavelet detail coefficients. Wavelets = time-frequency localization; FFT = global frequency only.

14 · hanr_wavelet() – Multiresolution Analysis

```
library(daltoolbox)
library(harbinger)

data(examples_anomalies)
dataset <- examples_anomalies$simple

har_plot(harbinger(), dataset$serie)

# Configurar detector Wavelet
# (usa MODWT internamente)
model <- hanr_wavelet()
model <- fit(model, dataset$serie)

detection <- detect(model,
                    dataset$serie)

print(detection |>
      dplyr::filter(event == TRUE))
## idx event type
## 1 50 TRUE anomaly
```

```
evaluation <- evaluate(model,
                      detection$event, dataset$event)
print(evaluation$confMatrix)
## detection TRUE FALSE
##   TRUE  1   0
##   FALSE 0 100

har_plot(model, dataset$serie,
          detection, dataset$event)

har_plot(model,
          attr(detection, "res"),
          detection, dataset$event,
          yline = attr(detection,
                      "threshold"))
```

Step by step

- Uses MODWT (Maximal Overlap Discrete Wavelet Transform)
- Decomposes the series into multiresolution detail bands
- Detail coefficients are aggregated to form a magnitude signal
- Large localized energy indicates anomalies
- Thresholds via `harutils()`; result: 1 TP, 0 FP

📌 **Key concept:** Wavelets decompose the series into multiple resolutions. Anomalies appear as elevated localized energy in high-frequency detail coefficients. Ref: Mallat, IEEE TPAMI 1989.

Summary: Fundamentals & Residual Models

Fundamentals

`hanr_histogram()`

First baseline for any new dataset. No model parameters — focuses on the empirical distribution. Fast and interpretable. Best used before trying any model-based approach.

Residual Models

`hanr_arima()`, `hanr_fbiad()`, `hanr_emd()`, `hanr_fft()` (+ AMOC, BinSeg, CUSUM, SMA variants), `hanr_garch()`, `hanr_remd()`, `hanr_rtad()`, `hanr_wavelet()`

Use when the series has a **known temporal structure**. Each method fits a different model (statistical, spectral, decomposition, or volatility-based) and flags observations whose residuals exceed a threshold. Most achieved 1 TP, 0 FP on the simple dataset — REMD produced 1 FP due to its combined EMD+ARIMA sensitivity.

References & Additional Sources

Academic references cited throughout the deck, and online resources to explore Harbinger further.

Online Resources

CRAN Package – harbinger

cran.r-project.org/web/packages/harbinger

Event Detection Resources

eic.cefet-rj.br/~eogasawara/en/event-detection-in-time-series

DAL Toolbox

cran.r-project.org/web/packages/daltoolbox

Issues & Discussions

github.com/cefet-rj-dal/harbinger/issues

Open questions, report bugs, or suggest improvements.

Key Academic References

- Huang et al. (1998). The empirical mode decomposition and the Hilbert spectrum. *Proc. R. Soc. London A*.
- Engle, R.F. (1982). Autoregressive conditional heteroscedasticity. *Econometrica*, 50(4).
- Bollerslev, T. (1986). Generalized autoregressive conditional heteroscedasticity. *J. Econometrics*, 31(3).
- Mallat, S. (1989). A theory for multiresolution signal decomposition. *IEEE TPAMI*, 11(7).
- Sakurada, M. & Yairi, T. (2014). Anomaly detection using autoencoders. *MLSDA Workshop*.
- Kingma, D.P. & Welling, M. (2013). Auto-encoding variational Bayes. *arXiv:1312.6114*.
- Makhzani, A. et al. (2016). Adversarial autoencoders. *arXiv:1511.05644*.
- Lima, T. et al. (2022). FBIAD: Forward-backward inertia anomaly detection. *IJCNN 2022*.
- Souza, V. et al. (2024). REMD: Robust EMD with ARIMA. *IJCNN 2024*.
- Jolliffe, I.T. (2002). Principal Component Analysis. 2nd ed. *Springer*.